

Workforce Prompt Pack

Markdown instruction files and implementation prompts prepared for Copilot and Replit, packaged into backend and frontend bundles.

Contents: merged repository Copilot instructions, backend implementation prompt, Replit project guide, and Replit agent prompt.

PDF note: This PDF was generated without encryption so text remains selectable and copyable.

Package Map

Two zip bundles were prepared with the same **prompts.pdf** included in both. The backend bundle contains the Copilot files under **.github/**. The frontend bundle contains the Replit files at the project root.

Backend package

- .github/copilot-instructions.md
- .github/prompts/backend-demo-slice.prompt.md
- prompts.pdf

Frontend package

- replit.md
- replit-agent-prompt.md
- prompts.pdf

1. Repository Copilot Instructions

File: .github/copilot-instructions.md

Merged repository-wide guidance plus Workforce-specific backend instructions for the demo slice.

```
# Copilot instructions for this repository
```

```
This file gives Copilot repository-specific guidance for build, test, lint, runtime setup, workspace conventions, and
```

```
When working inside a more specific subproject that has its own `.github/copilot-instructions.md` or `AGENTS.md`, pr
```

```
---
```

```
## Build, test, and lint commands
```

```
Notes:
```

- This workspace contains multiple related services rather than a single monolith
- Most backend services are Python FastAPI apps
- Frontends may be Next.js or Vite-based and live in project subfolders

```
### Common backend setup
```

```
Create and activate a virtual environment:
```

```
```bash
python -m venv venv
source venv/bin/activate
```
```

```
Install package and dev extras:
```

```
```bash
pip install -e .
pip install -e '.[dev]'
```

```
```
```

Apply database migrations:

```
```bash
alembic upgrade head
```
```

Seed demo data when supported:

```
```bash
python -m app.cli.main seed-demo
```
```

Run a backend service locally:

```
```bash
uvicorn app.main:app --host 0.0.0.0 --port 8000
```
```

Use `--reload` for iterative development when appropriate.

```
### Tests
Run the full test suite:
```

```
```bash
pytest -q
```
```

Run a targeted test file or function:

```
```bash
pytest tests/test_matching.py::test_preferred_comes_first -v
pytest path/to/test_file.py::test_name -q
```
```

```
### Frontend setup
Install and run:
```

```
```bash
npm install
npm run dev
```
```

Some frontend apps may use pnpm. Check `package.json` for the declared package manager and use that when appropriate

Run Playwright E2E tests when present:

```
```bash
npm run test:e2e
```
```

or:

```
```bash
npx playwright test
```
```

Generate TypeScript types from OpenAPI when present:

```
```bash
openapi-typescript $OPS/openapi.json -o src/lib/ops.types.ts
```
```

```
### Container / infra
Start the local stack:
```

```
```bash
docker compose up -d
```
```

```
### Linting / formatting
Use Ruff when available:
```

```
```bash
ruff check .
ruff --fix .
```
```

```
### Logs and runtime helpers
Background services may be started with `nohup`.
```

Logs are often written under `~/logs/` with names such as:

- `workforce\_8002.log`
- `hospitable\_ops\_8003.log`

Service control helper scripts may also exist in `~/bin`.

```
## High-level architecture
```

This workspace contains multiple related projects rather than one application.

```
### Common services
```

- **Workforce**: Python FastAPI backend providing workforce, scheduling, matching, and administration logic
- **Hospitable Ops**: Python FastAPI backend for operational workflows and internal tooling
- **Hospitable Web**: frontend application that consumes operational APIs and may use generated OpenAPI TypeScript types

```
### Development workflow
```

- Each service typically uses its own isolated Python virtual environment
- Backends are installed with editable installs
- Database schema changes are managed with Alembic
- Local backend development usually runs through Uvicorn
- Tests use Pytest
- Frontends may consume generated OpenAPI client types

```
### Repo organization conventions
```

This workspace may include multiple project trees such as:

- `PROJECTS\_CLEAN`
- `PROJECTS\_ARCHIVE`
- `projects\_active`

Subprojects may include their own:

- `.github/copilot-instructions.md`
- `AGENTS.md`

Always inspect those when working in a specific service area.

```
## Key conventions and non-obvious patterns
```

```
### Canonical project root environment variables
```

Scripts and docs may reference canonical environment variables instead of literal paths. Prefer them when present.

- `OPS\_HOME` – canonical path for hospitable-ops
- `WEB\_HOME` – canonical path for hospitable-web
- `WORKFORCE\_HOME` – canonical path for scheduler/workforce

```
### Packaging and dev extras
```

Use lean production installs and separate dev tooling:

```
```bash
```

```
pip install -e .
pip install -e '[dev]'
```
```

Migrations and demo seeding
Apply schema changes first:

```
```bash
alembic upgrade head
```
```

Then run service-specific demo seeding when supported:

```
```bash
python -m app.cli.main seed-demo
```
```

Naming and logs
Background services often write logs under `~/logs`.

Service control scripts may exist in `~/bin`, for example:

- `services\_start`
- `services\_stop`

Multiple instruction files may exist
When working inside a specific service, look for more local instructions before making changes.

Where to look next

Read service-level Copilot instructions when working inside a specific project:

- `workforce/.github/copilot-instructions.md`
- `workforce\_api/.github/copilot-instructions.md`
- `projects\_active/.github/copilot-instructions.md`
- `packages/\*/workforce/.github/copilot-instructions.md`

Also check `AGENTS.md` files in subprojects for automation-specific guidance.

Edit guidance for Copilot sessions

- Prefer running commands from the service root
- Use canonical environment variables when present
- When changing migrations or database schemas, include:
 - migration steps
 - seeding steps
 - test steps
- If the full test suite is slow, prefer targeted tests first
- Preserve service-specific patterns and avoid broad cross-project refactors unless explicitly requested

Service-specific instructions for Workforce backend in `projects\_active`

You are working on the Workforce backend for a demo-ready slice that proves:

1. Workforce user administration
2. Hospitable room and task operations inside Workforce

Product boundaries

Workforce is the system of record for

- businesses
- locations

- users
- roles
- permissions
- scoped role assignments

Hospitable owns

- rooms / units
- room status
- tasks
- inspections
- issues
- event logs

Do not build in this slice

- payroll
- timeclock
- advanced scheduling
- CRM features
- messaging / chat
- duplicated workforce scheduling logic inside hospitable workflows
- external integrations
- advanced offline sync

Technical rules

- Preserve the existing FastAPI + SQLAlchemy + Alembic architecture
- Reuse current project structure, naming patterns, and established conventions
- Keep OpenAPI and Pydantic schemas typed and explicit
- Prefer additive changes over broad refactors
- Do not introduce a second ORM or a second API framework
- Keep migrations small and reviewable
- Add tests for each new route and service slice
- Keep responses stable and frontend-friendly

Authorization model

- Permissions come only from roles
- Roles are BUSINESS-scoped or LOCATION-scoped
- A user can have different roles at different locations
- Job title is display-only and belongs on the user's role assignment, not in the authorization model
- Enforce scope validation in service logic and schema validation where possible
- Reject invalid cross-business and cross-location assignments

Required backend entities

Build or complete only what is needed for the demo slice:

- Business
- Location
- User
- Role
- Permission
- UserRoleAssignment
- Room
- Task
- Inspection
- Issue
- EventLog

Required backend capabilities

User administration

- list users
- create users
- update users
- assign scoped roles
- remove scoped roles
- list roles with permissions
- return effective permissions when practical
- return user assignments in a frontend-friendly shape

Hospitable

- list rooms by location
- create rooms
- update rooms
- update room status
- create tasks
- list tasks by location and status
- assign tasks to users
- update task status
- record inspection result
- create issue / maintenance hold
- write event log entries for major actions

Dashboard

Build only enough to support a location summary endpoint with:

- room counts by status
- task counts by status
- optional recent activity if easy to include

Demo-first priorities

Build only enough to support this demo flow:

1. Admin creates or updates a user
2. Admin assigns a role at business or location scope
3. Manager views rooms in a location
4. Manager creates or assigns a cleaning task
5. Housekeeper marks task in progress and completed
6. Inspector records inspection
7. Dashboard reflects counts by location and status

Data model expectations

UserRoleAssignment

Must support at minimum:

- `id`
- `user\_id`
- `business\_id`
- `scope\_type` enum: `BUSINESS` or `LOCATION`
- `location\_id` nullable only for `BUSINESS` scope
- `role\_id`
- `job\_title\_label` display-only
- timestamps if consistent with project patterns

Validation requirements:

- BUSINESS scope assignments must not require `location\_id`
- LOCATION scope assignments must require `location\_id`
- `location\_id` must belong to `business\_id`
- role scope must match assignment scope

Room

Should support at minimum:

- `id`
- `business\_id`
- `location\_id`
- `name` or `room\_number`
- `sector` optional
- `status`
- `floor\_type` optional
- `bed\_summary` optional
- `notes` optional

Task

Should support at minimum:

- `id`
- `business\_id`
- `location\_id`
- `room\_id`

- `task\_type`
- `status`
- `assigned\_user\_id` nullable
- `priority` optional
- `due\_at` nullable
- `notes` optional

Status conventions

Room statuses

Use enums or centralized constants:

- `clean`
- `dirty`
- `ready\_for\_inspection`
- `inspected`
- `stayover`
- `dnd`
- `laundry\_only`
- `maintenance\_hold`

Task statuses

Use enums or centralized constants:

- `open`
- `assigned`
- `in\_progress`
- `completed`
- `blocked`
- `cancelled`

Delivery expectations

When implementing work:

- inspect existing models, routes, and services first
- follow current repo patterns before inventing new structure
- make the smallest complete vertical slice
- include migration, schemas, route wiring, service logic, and tests
- avoid placeholder stubs unless absolutely necessary
- do not leave TODO comments for core requested logic if it can be completed now

Quality bar

- validate business/location scope relationships
- reject invalid cross-location assignments
- use enums/constants for status fields
- include timestamps where existing patterns support them
- ensure responses are frontend-friendly
- add seed/demo data support where helpful

Preferred sequence

1. RBAC and scoped user administration
2. Room model and room endpoints
3. Task model and assignment/status endpoints
4. Inspection and issue endpoints
5. Dashboard summary endpoint
6. Seed/demo data and tests

API targets for frontend alignment

Aim to support these endpoints or the closest equivalent matching existing repo conventions:

Users / RBAC

- `GET /users`
- `GET /roles`
- `POST /users/{id}/assignments`
- `DELETE /users/{id}/assignments/{assignment\_id}`

Rooms

- `GET /locations/{location\_id}/rooms`
- `POST /locations/{location\_id}/rooms`
- `PATCH /rooms/{room\_id}`

Tasks

- `GET /locations/{location\_id}/tasks`
- `POST /locations/{location\_id}/tasks`
- `POST /tasks/{task\_id}/assign`
- `POST /tasks/{task\_id}/status`

Dashboard

- `GET /locations/{location\_id}/dashboard-summary`

Priority of instructions

When multiple instruction sources exist, apply them in this order:

1. More local `.github/copilot-instructions.md` in the service being edited
2. Relevant `AGENTS.md` in the service being edited
3. This repository-wide instruction file

If instructions conflict, prefer the more local file.

Last updated: 2026-03-25

2. Backend Demo Slice Prompt

File: .github/prompts/backend-demo-slice.prompt.md

Primary implementation prompt for GitHub Copilot to build the Workforce backend demo slice.

```
# Workforce Backend Demo Slice
```

```
Implement the backend needed for a demo-ready slice of Workforce that proves:
```

1. user administration with scoped roles
2. hospitable room and task operations

```
## Goal
```

```
Build the smallest complete backend slice needed to support a live frontend demo.
```

```
## Technical constraints
```

- Reuse the existing FastAPI, SQLAlchemy, Alembic, and schema structure already in the repo
- Follow current naming and routing conventions
- Make incremental changes
- Do not refactor unrelated modules
- Add tests for core behavior

```
## Required scope: phase 1
```

```
Implement or complete the user administration slice first.
```

```
### Required models / support
```

- Role
- Permission
- UserRoleAssignment

```
### UserRoleAssignment requirements
```

```
Support these fields:
```

- user_id
- business_id
- scope_type enum: BUSINESS or LOCATION
- location_id nullable only for BUSINESS scope
- role_id
- job_title_label display-only

```
### Required routes
```

```
Add or complete routes to:
```

- list users with their scoped assignments
- assign a role to a user
- remove a role assignment from a user
- list roles with permissions

```
### Required validation
```

- enforce scope validation
- reject invalid business/location combinations
- reject invalid role-to-scope combinations
- ensure location belongs to business where applicable

```
### Required deliverables
```

- model updates or new models
- schemas
- service logic
- routes
- migration(s)
- tests

```
## Required scope: phase 2
```

```
After the RBAC slice is complete, implement the hospitable room/task slice.
```

```
### Required models / support
```

- Room

- Task
- EventLog

Room fields

At minimum:

- business_id
- location_id
- name or room_number
- sector optional
- status enum
- floor_type optional
- bed_summary optional
- notes optional

Task fields

At minimum:

- business_id
- location_id
- room_id
- task_type
- status enum
- assigned_user_id nullable
- priority optional
- due_at nullable
- notes optional

Required routes

Add or complete routes to:

- list rooms by location
- create rooms
- update rooms
- update room status
- list tasks by location and status
- create task
- assign task
- update task status

Event logging

Write event log entries when:

- task is created
- task is assigned
- task status changes
- room status changes

Required deliverables

- model updates or new models
- schemas
- service logic
- routes
- migration(s)
- tests

Demo flow to support

The final backend should support this exact flow:

1. Admin creates or updates a user
2. Admin assigns a role at business or location scope
3. Manager views rooms in a location
4. Manager creates or assigns a cleaning task
5. Housekeeper marks task in progress and completed
6. Inspector records inspection
7. Dashboard returns counts by location and status

Keep out of scope

Do not add:

- payroll
- timeclock
- advanced scheduling

- messaging
- CRM modules
- external integrations
- advanced offline sync

API target shape

Aim to support these endpoints or the closest equivalent matching repo conventions:

Users / RBAC

- GET /users
- GET /roles
- POST /users/{id}/assignments
- DELETE /users/{id}/assignments/{assignment_id}

Rooms

- GET /locations/{location_id}/rooms
- POST /locations/{location_id}/rooms
- PATCH /rooms/{room_id}

Tasks

- GET /locations/{location_id}/tasks
- POST /locations/{location_id}/tasks
- POST /tasks/{task_id}/assign
- POST /tasks/{task_id}/status

Dashboard

- GET /locations/{location_id}/dashboard-summary

Important

Implement the code. Do not return only a plan.

3. Replit Project Guide

File: replit.md

Project-level guidance for building the Workforce frontend demo in Replit.

```
# Workforce Frontend Demo
```

```
## Goal
```

Build a demo-ready frontend for Workforce that proves:

1. Workforce user administration
2. Hospitable room and task operations

This frontend is for demonstration, but it should still feel like a real internal operations product.

```
## Recommended stack
```

- React
- TypeScript
- Vite

```
## Design goals
```

- dark mode first
- clean operational dashboard feel
- responsive for desktop and tablet
- fast, simple workflows
- reusable components
- clear business and location context throughout the app

```
## Product structure
```

```
### Workforce owns
```

- user administration
- roles
- permissions
- business context
- location context

```
### Hospitable inside Workforce owns
```

- room views
- room status
- task views
- task assignment
- task status updates
- inspections and issues if backend is ready

```
## Required screens
```

```
### 1. Dashboard
```

Show:

- room counts by status
- task counts by status
- selected business and location
- optional recent activity list if available

```
### 2. Users
```

Show:

- user table
- scoped role assignments
- job title label as display-only
- role assignment modal or drawer

```
### 3. Rooms
```

Show:

- room list or grid by selected location
- status badges

- room details drawer or modal
- simple room edit capability if backend supports it

4. Tasks

Show:

- task list grouped by status
- assign task action
- update task status action
- room reference and assignee display

App shell

Create:

- top navigation
- sidebar
- business selector
- location selector

The selected business and location should drive the data shown on dashboard, rooms, and tasks screens.

Technical rules

- Use environment variable `VITE\_API\_BASE\_URL`
- Put API calls in a dedicated API client layer
- Use typed interfaces for server responses
- Use reusable UI components
- Keep routing simple and clear
- Add loading, empty, and error states
- Prefer maintainable structure over over-engineering

Mock mode requirement

If some backend endpoints are not ready yet:

- preserve the real API shapes
- implement a mock adapter with the same response structure
- make it easy to switch between mock and live API

Suggested approach:

- `src/lib/api/` for live API functions
- `src/lib/mock/` for mock data and mock service functions
- `src/config/` for environment and feature toggles

Data contract targets

Users / RBAC

- GET /users
- GET /roles
- POST /users/{id}/assignments
- DELETE /users/{id}/assignments/{assignment_id}

Rooms

- GET /locations/{location_id}/rooms
- POST /locations/{location_id}/rooms
- PATCH /rooms/{room_id}

Tasks

- GET /locations/{location_id}/tasks
- POST /locations/{location_id}/tasks
- POST /tasks/{task_id}/assign
- POST /tasks/{task_id}/status

Dashboard

- GET /locations/{location_id}/dashboard-summary

Visual rules

- dark background
- clear elevated cards
- readable tables
- compact but not cramped spacing
- obvious status color treatment

- admin-product feel, not consumer app feel

Suggested folder structure

- `src/app/`
- `src/components/`
- `src/features/dashboard/`
- `src/features/users/`
- `src/features/rooms/`
- `src/features/tasks/`
- `src/lib/api/`
- `src/lib/mock/`
- `src/context/`
- `src/types/`

Build order

1. app shell and routing
2. business and location context
3. dashboard page
4. users page
5. rooms page
6. tasks page
7. mock/live API toggle cleanup

Demo data expectations

If mock mode is used, seed:

- 1 business
- 2 locations
- 6 users
- 10 to 20 rooms
- 8 to 12 active tasks
- mixed room statuses
- mixed task statuses
- 1 maintenance hold
- 1 failed inspection if inspection support is present

Final expectation

The demo should support this flow:

1. Admin views users
2. Admin assigns a scoped role
3. Manager views rooms for a location
4. Manager assigns a room task
5. Housekeeper moves task to in progress and then completed
6. Dashboard reflects the result

4. Replit Agent Prompt

File: replit-agent-prompt.md

Direct prompt for Replit Agent to scaffold and implement the frontend demo.

```
# Replit Agent Prompt
```

```
Build the Workforce frontend demo as a React + TypeScript + Vite application.
```

```
## Goal
```

```
Create a dark-mode operations dashboard that demonstrates both Workforce user administration and Hospitable room/tasks
```

```
## Requirements
```

- Create a clean app shell with:
 - top navigation
 - sidebar
 - business selector
 - location selector
- Create routes/pages for:
 - Dashboard
 - Users
 - Rooms
 - Tasks

```
### Users page
```

- user table
- role assignment UI
- display scope as Business or Location
- show job title label as display-only

```
### Rooms page
```

- list or grid of rooms by selected location
- colored status badges
- detail drawer or modal

```
### Tasks page
```

- list grouped by status
- assign task action
- update task status action

```
### Dashboard
```

- summary cards for room statuses and task statuses
- recent activity list if possible

```
## Technical constraints
```

- Use React + TypeScript + Vite
- Use environment variable VITE_API_BASE_URL
- Put API calls in a dedicated client layer
- Use typed interfaces for server responses
- Build with reusable UI components
- Keep the design dark, simple, and operational
- Do not build unrelated features

```
## Demo mode
```

- If backend endpoints are missing, create a temporary mock adapter layer with the same shapes as the intended API spec
- Make it easy to swap mock mode for live API later

```
## Deliverables
```

- runnable frontend
- clean folder structure
- reusable components
- sample mock data if needed
- clear place to configure backend URL